

Overview

A robust system designed to optimize and streamline the management of insurance claims, resource allocation, and client communication, ensuring efficient processing and settlement of claims.

Users of the System

Claims Adjusters: Evaluate claims, manage resources, and interact with policyholders.

Resource Coordinators: Handle allocation of adjusters, appraisers, and other resources to claims.

Policyholders: Submit claims, provide necessary documentation, and receive status updates.

Functional Requirements

User Registration & Profile Management: Users create accounts, log in, and manage their profiles with stringent data validation and secure information handling.

Claims Processing & Management: Adjusters initiate, update, and manage claim details, with proper authorization for any changes.

Resource Allocation to Claims: Coordinators allocate resources such as adjusters, investigators, and legal counsel to specific claims.

Policyholder Interaction Interface: Policyholders can track claim status and communicate with claims adjusters.

User Role-Based Authentication: The system assigns user roles to provide tailored interfaces and functionality.

JWT Authorization: Manages user sessions and secures API calls with JWT tokens.

RESTful API & Angular Service Layer: Enables efficient data exchange and UI updates between the Angular frontend and RESTful backend services.

Technology Stack

Backend: Spring Boot, JPA, MySQL

Frontend: Angular

Security: Spring Security, JWT

Key Points to Note

Security: Prioritize the protection of claimant data and API access.

Scalability: Ensure the platform can handle a growing number of claims and users.

User Interface Consistency: Maintain a consistent and user-friendly interface across all modules.

Best Practices: Commit to high coding standards for maintainability and readability.

Backend Functionalities to be Built

User Management: Develop endpoints for user registration, login, and profile management.

Claims Management: Implement CRUD operations for claim details, maintaining data integrity.

Resource Management: Monitor and assign resources efficiently to manage claims.

Role-Based Authentication: Establish clear access levels for adjusters, coordinators, and policyholders.

JWT Token Management: Facilitate secure token generation, validation, and expiration.

Backend Test Cases

User Registration: Confirm secure storage and handling of user data.

Claim Initiation: Ensure that claims are correctly created with all required details.

Resource Allocation: Test the precise allocation and update of claim resources.

Login Authentication: Verify the authentication process for accuracy and security

JWT Token: Check JWT token generation and expiration for compliance with security standards.

Your task is to complete the following backend files:

/src/main/java/com/wecp/insuraceclaimssystem/config/SecurityConfig.java

/src/main/java/com/wecp/insuraceclaimssystem/controller/AdjusterController.java

/src/main/java/com/wecp/insuraceclaimssystem/controller/CoordinatorController.java

./src/main/java/com/wecp/insuraceclaimssystem/controller/PolicyHolderController.java

/src/main/java/com/wecp/insuraceclaimssystem/controller/RegisterAndLoginController.java

/src/main/java/com/wecp/insuraceclaimssystem/entity/Claim.java

./src/main/java/com/wecp/insuraceclaimssystem/entity/Resource.java

./src/main/java/com/wecp/insuraceclaimssystem/entity/User.java

./src/main/java/com/wecp/insuraceclaimssystem/jwt/JwtRequestFilter.java

./src/main/java/com/wecp/insuraceclaimssystem/jwt/JwtUtil.java

./src/main/java/com/wecp/insuraceclaimssystem/repository/ClaimRepository.java

./src/main/java/com/wecp/insuraceclaimssystem/repository/ResourceRepository.java

/src/main/java/com/wecp/insuraceclaimssystem/repository/UserRepository.java

/src/main/java/com/wecp/insuraceclaimssystem/service/ClaimService.java

/src/main/java/com/wecp/insuraceclaimssystem/service/ResourceService.java

/src/main/java/com/wecp/insuraceclaimssystem/service/UserService.java

API endpoints:

For Claims Adjusters (Admin Side):

Register Adjuster: POST /api/user/register

Login Adjuster: POST /api/user/login

Create Claim: POST /api/policyholder/claim

View Claims: GET /api/policyholder/claims

For Resource Coordinators and Policyholders (User Side):

Register User: POST /api/user/register

Login User: POST /api/user/login

View Assigned Resources: GET /api/coordinator/resources

Update Claim Progress: PUT /api/coordinator/update-claim

View Claim Status: GET /api/policyholder/claim-status

Frontend Functionalities to be Built

Registration and Profile Management: Streamlined interfaces for all user types.

Claims Dashboard: Tools for adjusters to manage claims and necessary resources.

Resource Allocation Interface: System for coordinators to assign resources to new and existing claims.

Policyholder Communication Interface: Portal for policyholders to file claims and communicate with adjusters.

Role-Specific UI Elements: Customized UI based on the user's role

Session Management with JWT: Robust JWT token handling for session security.

Your task is to complete the following frontend files:

/src/app/add-resource/add-resource.component.ts

/src/app/add-resource/add-resource.component.html

/src/app/create-claim/create-claim.component.ts

/src/app/create-claim/create-claim.component.htm

/src/app/view-claim-status/view-claim-status.component.ts

/src/app/view-claim-status/view-claim-status.component.html

./src/app/view-resource/view-resource.component.ts

/src/app/view-resource/view-resource.component.html

/src/app/resource-allocate/resource-allocate.component.ts

/src/app/resource-allocate/resource-allocate.component.html

/src/app/login/login.component.ts

/src/app/login/login.component.html

/src/services/http.service.ts

./src/app/app.component.html

/src/app/registration/registration.component.ts

/src/app/registration/registration.component.html